

Package ‘indexDesignWindows’

April 29, 2026

Type Package

Title Replication Package: Redesigning Historical Windows in Index-Based Insurance

Version 0.0.0.9000

Author Francis Tsiboe [aut, cre] (<<https://orcid.org/0000-0001-5984-1072>>)

Maintainer Francis Tsiboe <ftsiboe@hotmail.com>

Creator Francis Tsiboe

Description Replication package for a study evaluating alternative historical window designs used to construct the Pasture, Rangeland, and Forage (PRF) rainfall index under the U.S. Federal Crop Insurance Program (FCIP). The package systematically compares index designs based on varying lengths of historical climate data to assess implications for index stability, spatial equity, indemnity accuracy, and policy performance. It provides reproducible workflows, pre-processed outputs, and visualization tools to support robustness analysis of index-based insurance products in both U.S. and international contexts.

License GPL-3 + file LICENSE

URL <https://github.com/ftsiboe/indexDesignWindows>

BugReports <https://github.com/ftsiboe/indexDesignWindows/issues>

Encoding UTF-8

Roxygen list(markdown = TRUE)

RoxygenNote 7.3.3

VignetteBuilder knitr

Depends R (>= 4.1.0)

Imports data.table, ggplot2, ggrepel, gridExtra, gtable, sf, urbnmapr

Remotes github::ftsiboe/rfcipDemand, github::UrbanInstitute/urbnmapr

Suggests dplyr, tidyr, knitr, rmarkdown, withr, stats, rfcipDemand, piggyback, classInt, test-that (>= 3.0.0), lmtest, plm, stringr, terra

LazyData true

Cite-us If you find it useful, please consider starring the repository and citing the following studies

- Tsiboe, F. and Turner, D. (2025). ``Incorporating buy-up price loss coverage into the United States farm safety net." Applied Economic Perspectives and Policy.
- Tsiboe, F., et al. (2025). ``Risk reduction impacts of crop insurance in the United States." Applied Economic Perspectives and Policy.
- Gaku, S. and Tsiboe, F. (2024). Evaluation of alternative farm safety net program combination strategies. Agricultural Finance Review.

Contents

add_break_categories	2
classify_mean_variance_trends	3
classify_trend	4
compute_balance	5
ers_theme	7
estimate_by_estimator	7
estimate_final_turning_point	8
estimate_one_state	10
estimate_one_state_fgls	11
estimate_turning_point_benchmark	12
extract_model_diagnostics	14
fisher_joint_pvalue	15
get_official_prf_polygon	15
get_price_indices	16
get_study_releases	17
plot_fcip_main_outcomes	18
plot_liability_and_acres	20
plot_prf_statistics	22
plot_us_states_choropleth	23
policytime	25
reshape_statistics	26
select_tolerance_by_post_stability	27
setup_environment	28
Index	30

add_break_categories	<i>Add Categorical Breaks to a Numeric Variable</i>
----------------------	---

Description

Classifies a numeric variable into categorical intervals using either Fisher-Jenks natural breaks (via **classInt**) or user-specified break points, and appends the resulting factor variable to the input data.

Usage

```
add_break_categories(  
  data,  
  variable,  
  break_n = NULL,  
  break_levels = NULL,  
  break_labels = NULL  
)
```

Arguments

data	A data frame containing the variable to be classified.
variable	Character scalar giving the name of the numeric column in data to classify.
break_n	Integer number of classes to compute when using Fisher-Jenks breaks. Required when break_levels is NULL.

break_levels	Optional numeric vector of break points. If supplied, these are used directly and break_n is ignored.
break_labels	Optional character vector of labels to use for the resulting factor. If NULL, labels are generated from break_levels.

Details

The function supports two main workflows:

- **Automatic breaks:** If break_levels is NULL, Fisher-Jenks natural breaks are computed using `classInt::classIntervals()` with `style = "fisher"`. In this case, break_n must be supplied.
- **Manual breaks:** If break_levels is provided, these are used directly and break_n is ignored. If break_labels is NULL, labels are generated from break_levels in the form "a - b" (two decimal places).

Intervals are constructed with `include.lowest = TRUE` and `right = FALSE`, so they are left-closed and right-open.

Value

The original data with an additional factor column named `<variable>_cat` containing the categorical breaks.

```
classify_mean_variance_trends
```

Classify mean and variance trends jointly

Description

Creates statistical and descriptive classifications for county-level mean and variance trend estimates across one or more significance thresholds.

Usage

```
classify_mean_variance_trends(
  res,
  mean_estimate_col = "mean_estimate",
  mean_p_value_col = "mean_p_value",
  var_estimate_col = "var_estimate",
  var_p_value_col = "var_p_value",
  sig_levels = c(0.01, 0.05, 0.1)
)
```

Arguments

res	A data.table containing coefficient estimates and p-values.
mean_estimate_col	Name of the column containing mean trend estimates. Default is "mean_estimate".
mean_p_value_col	Name of the column containing mean trend p-values. Default is "mean_p_value".

var_estimate_col Name of the column containing variance trend estimates. Default is "var_estimate".

var_p_value_col Name of the column containing variance trend p-values. Default is "var_p_value".

sig_levels Numeric vector of significance levels. Default is `c(0.01, 0.05, 0.10)`.

Details

The component classifications are based on separate mean and variance tests. For example, a county may be classified as "Positive Mean only" if the mean trend is positive and statistically significant while the variance trend is not statistically significant.

Fisher's method is reported separately as a combined-evidence test across the mean and variance p-values. The final `trend_class_XX` columns are intended for interpretation, mapping, and summary tables.

Value

A copy of `res` with additional classification columns:

joint_fisher_stat Fisher combined test statistic.

joint_fisher_p_value Fisher combined p-value.

mean_trend_class_XX Mean trend class at significance level XX.

var_trend_class_XX Variance trend class at significance level XX.

joint_trend_class_XX Descriptive mean-variance pattern.

joint_fisher_class_XX Whether Fisher's combined p-value indicates joint evidence at significance level XX.

trend_class_XX Clean final classification for mapping and summary tables.

See Also

Other Precipitation trend analysis: [classify_trend\(\)](#), [estimate_by_estimator\(\)](#), [estimate_one_state\(\)](#), [estimate_one_state_fgl\(\)](#), [extract_model_diagnostics\(\)](#), [fisher_joint_pvalue\(\)](#)

<code>classify_trend</code>	<i>Classify a trend by sign and statistical significance</i>
-----------------------------	--

Description

Classifies a coefficient estimate as positive, negative, or not significant based on its sign and p-value.

Usage

```
classify_trend(estimate, p_value, p_value_sig = 0.05)
```

Arguments

estimate Numeric vector of coefficient estimates.

p_value Numeric vector of p-values associated with the estimates.

p_value_sig Numeric significance threshold. Default is `0.05`.

Value

A character vector with values:

"Positive" Estimate is positive and statistically significant.

"Negative" Estimate is negative and statistically significant.

"None" Estimate is not statistically significant or equals zero.

NA Estimate or p-value is missing.

See Also

Other Precipitation trend analysis: [classify_mean_variance_trends\(\)](#), [estimate_by_estimator\(\)](#), [estimate_one_state\(\)](#), [estimate_one_state_fgls\(\)](#), [extract_model_diagnostics\(\)](#), [fisher_joint_pvalue\(\)](#)

compute_balance	<i>Distributional balance: one basis vs multiple comparison variables (long format)</i>
-----------------	---

Description

For a given **basis** numeric column (`col_x`) and one or more **comparison** numeric columns (`col_y`), this function computes a rich set of diagnostics describing bias, scale, agreement, and distributional differences between the basis and each comparison variable.

Results are returned in **long format** with one row per group (defined by `by`) and per comparison variable (`y_level`).

Metrics include:

- **Sample size & moments**

- `n`: number of non-missing pairs
- `mean_x`, `mean_y`
- `var_x`, `var_y`
- `sd_x`, `sd_y`
- `cv_x`, `cv_y` (coefficients of variation)

- **Bias & scale**

- `mean_bias` = `mean_x` - `mean_y`
- `mae` = `mean(abs(x - y))`
- `rmse` = `sqrt(mean((x - y)^2))`
- `smape` = `2 * mean(abs(x - y) / (abs(x) + abs(y)))`, with safe handling when denominator is zero
- `ratio_means` = `mean_x / mean_y`
- `cv_ratio` = `cv_x / cv_y`
- `mpd` = mean percentage difference:

$$(((\bar{x} - \bar{y}) / \bar{y}) - 1) * 100$$

- **Balance / distribution comparison**

- `msd` = mean standardized difference: `(mean_x - mean_y) / sqrt((sd_x^2 + sd_y^2) / 2)`
- `var_ratio` = `var_x / var_y`
- `ks_stat` = Kolmogorov-Smirnov D-statistic (two-sample)

- emd = 1D Wasserstein / Earth Mover's Distance, approximated as mean absolute difference between quantiles of x and y
 - hellinger = Hellinger distance computed from common histograms
 - pvalue_mean = p-value for equality of means (paired t-test)
 - pvalue_ks = p-value from Kolmogorov-Smirnov test
 - pvalue_var = p-value from F-test of equal variances
 - pvalue_levene = p-value from Levene test (mean-based)
 - pvalue_brown_forsythe = p-value from Brown-Forsythe test
 - pvalue_kruskal_wallis = p-value from Kruskal-Wallis test
 - **Zero-mass / structural zeros**
 - zero_share_x = proportion of observations with x == 0
 - zero_share_y = proportion of observations with y == 0
 - zero_share_diff = zero_share_x - zero_share_y
 - pvalue_zero_mcnemar = p-value from McNemar test on the paired zero/non-zero indicators
 - **Bounded-scale / logit-space (for (0, 1) variables)**
 - mean_logit_x, mean_logit_y = means of logit-transformed x and y
 - mean_logit_diff = mean(logit_x - logit_y)
 - rmse_logit = sqrt(mean((logit_x - logit_y)^2))
 - pvalue_mean_logit = p-value from paired t-test on logit_x vs logit_y
- These logit-based metrics are computed only if all non-missing x and y are in the interval (0, 1). Otherwise, they are returned as NA.
- **Agreement / association**
 - pearson_cor = Pearson correlation
 - spearman_cor = Spearman rank correlation
 - kendall_cor = Kendall's tau
 - ccc = Lin's Concordance Correlation Coefficient
 - pvalue_pearson_cor
 - pvalue_spearman_cor
 - pvalue_kendall_cor
 - **Quantile / tail behavior**
 - Quantiles at p = 0.10, 0.25, 0.50, 0.75, 0.90 for x and y: q10_x, q25_x, q50_x, q75_x, q90_x q10_y, q25_y, q50_y, q75_y, q90_y
 - Differences: q10_diff, q25_diff, q50_diff, q75_diff, q90_diff computed as (q_x - q_y)

Usage

```
compute_balance(dt, col_x, col_y, by = NULL)
```

Arguments

dt	A data.frame or data.table.
col_x	Character scalar. Name of the basis numeric column.
col_y	Character vector. Names of one or more comparison numeric columns. The basis variable col_x is always compared against each variable in col_y.
by	Optional character vector of grouping variables. If NULL (default), all rows are treated as one group.

Value

A data.table in **long format** with columns: c(by, "y_level", "n", "mean_x", "mean_y", "var_x", "var_y", "sd_x", "sd_y", "cv_x", "cv_y", "cv_ratio", "mean_bias", "mae", "rmse", "smape", "ratio_means", "mpd", "msd", "var_ratio", "ks_stat", "emd", "hellinger", "pvalue_mean", "pvalue_ks", "pvalue_var", "pvalue_levene", "pvalue_brown_forsythe", "pvalue_kruskal_wallis", "zero_share_x", "zero_share_y", "zero_share_diff", "pvalue_zero_mcnemar", "mean_logit_x", "mean_logit_y", "mean_logit_diff", "rmse_logit", "pvalue_mean_logit", "pearson_cor", "spearman_cor", "kendall_cor", "ccc", "pvalue_pearson_cor", "pvalue_spearman_cor", "pvalue_kendall_cor", "q10_x", "q25_x", "q50_x", "q75_x", "q90_x", "q10_y", "q25_y", "q50_y", "q75_y", "q90_y", "q10_diff", "q25_diff", "q50_diff", "q75_diff", "q90_diff").

ers_theme	<i>ERS Theme</i>
-----------	------------------

Description

ERS Theme

Usage

```
ers_theme()
```

Source

copied from <https://github.com/USDA-REE-ERS/MTED-Theme> on 08/01/2025

Examples

```
ggplot2::ggplot() + ers_theme()
```

estimate_by_estimator	<i>Estimate county-level trends for one panel estimator</i>
-----------------------	---

Description

Applies the one-state estimation workflow separately by state for a given panel estimator. The function returns county-level mean and variance trend estimates for all states in the supplied data.

Usage

```
estimate_by_estimator(dtw, estimator)
```

Arguments

dtw	A data.table containing the rolling-window estimation sample. Must include state_code, panelid, commodity_year, precipitation, trend, and county_fips.
estimator	Character string specifying the plm model estimator to use. Examples include "pooling", "within", "random", "between", and "fd".

Details

The function estimates each state separately using:

```
precipitation ~ trend:factor(county_fips)
```

This produces county-specific trend coefficients within each state. If a state model fails, the function returns a structured empty row with missing values, allowing downstream `rbindlist()` calls to remain stable.

Value

A `data.table` with county-level trend estimates for the selected estimator. The output includes state identifiers, model estimator labels, mean trend estimates, variance trend estimates, and model diagnostics.

See Also

Other Precipitation trend analysis: [classify_mean_variance_trends\(\)](#), [classify_trend\(\)](#), [estimate_one_state\(\)](#), [estimate_one_state_fgls\(\)](#), [extract_model_diagnostics\(\)](#), [fisher_joint_pvalue\(\)](#)

```
estimate_final_turning_point
```

Estimate Final Turning Points Using a Data-Driven Tolerance

Description

This function runs the full benchmark-distance turning-point pipeline. It first selects the best tolerance value for each group using `select_tolerance_by_post_stability()`. It then re-estimates the final turning point for each group using the selected tolerance.

Usage

```
estimate_final_turning_point(
  data,
  outcome,
  group_var,
  x_var,
  benchmark_x = NULL,
  tolerance_grid = seq(0.01, 0.15, by = 0.005),
  consecutive_years = 5,
  penalty_weight = 0.01,
  require_consecutive_x = TRUE
)
```

Arguments

<code>data</code>	A <code>data.frame</code> or <code>data.table</code> containing the outcome variable, grouping variable, and historical-window variable.
<code>outcome</code>	Character string. Name of the outcome variable to evaluate. Examples include "actuarial_index", "alternative_lr", or "baseline_lr".
<code>group_var</code>	Character string. Name of the grouping variable used to estimate separate turning points. For example, "adjustment".

<code>x_var</code>	Character string. Name of the historical-window variable. This variable should represent the number of years in the historical window.
<code>benchmark_x</code>	Numeric. The x-value used as the benchmark. If NULL, the maximum observed value of <code>x_var</code> is used. For example, use 60 for a 60-year benchmark or 200 if the full sample is coded as 200.
<code>tolerance_grid</code>	Numeric vector. Candidate tolerance shares to evaluate. The default evaluates values from 1 percent to 15 percent in increments of 0.5 percentage points.
<code>consecutive_years</code>	Integer. Number of consecutive historical-window values that must remain within the benchmark-distance tolerance before the first x-value is classified as the turning point. The default is 5.
<code>penalty_weight</code>	Numeric. Penalty applied to later turning points in the score function. Larger values favor earlier turning points, while smaller values allow later turning points if they produce lower post-turning-point variability. The default is 0.01.
<code>require_consecutive_x</code>	Logical. If TRUE, the function requires the selected sequence to be consecutive in terms of the actual <code>x_var</code> values.

Details

The function returns both the final turning point and the diagnostics used to select the tolerance. This makes the final output easier to interpret and audit because each reported turning point is linked to the selected tolerance, post-turning-point stability statistics, and final score.

The full pipeline is:

1. For each tolerance in `tolerance_grid`, estimate the earliest benchmark- stable historical window for each group.
2. For each candidate tolerance and group, calculate the stability of the outcome after the estimated turning point.
3. Select the tolerance that minimizes:

$$score = post_cv + penalty_weight \times scaled_turning_point$$

1. Re-estimate the final turning point using the selected group-specific tolerance.

This function is useful when the analyst does not want to impose an arbitrary tolerance, such as 5 percent, and instead wants the tolerance to be selected from the data based on post-turning-point stability.

Value

A data.table with one row per group. The output includes the grouping variable, outcome name, benchmark year, selected tolerance share, required number of consecutive years, final turning point, post-turning-point stability statistics, scaled turning point, and minimized score.

estimate_one_state	<i>Estimate one state's mean and variance trend coefficients</i>
--------------------	--

Description

Runs the FGLS-style mean-variance workflow for one state and returns a tidy coefficient table containing county-level mean and variance trend estimates.

Usage

```
estimate_one_state(dt, formula, model = "within")
```

Arguments

<code>dt</code>	A <code>data.table</code> or data frame containing one state's panel data. Required columns include <code>panelid</code> , <code>commodity_year</code> , <code>precipitation</code> , <code>trend</code> , and <code>county_fips</code> .
<code>formula</code>	Formula for the mean trend model. In this workflow, the expected formula is <code>precipitation ~ trend:factor(county_fips)</code> .
<code>model</code>	Character string specifying the <code>plm</code> estimator.

Details

Robust coefficient tables are computed using `lmtest::coeftest()` with `plm::vcovDC()`. This is possible because the final mean model is estimated using pre-weighted variables rather than the `weights` argument in `plm`.

Value

A `data.table` with county-level trend estimates and diagnostics. The returned table includes:

- term** Coefficient term from the model.
- county_fips** County FIPS extracted from the model term.
- mean_estimate** Estimated mean trend coefficient.
- mean_standard_error** Robust standard error for the mean trend.
- mean_t_value** t-statistic for the mean trend.
- mean_p_value** p-value for the mean trend.
- var_estimate** Estimated variance trend coefficient.
- var_standard_error** Robust standard error for the variance trend.
- var_t_value** t-statistic for the variance trend.
- var_p_value** p-value for the variance trend.
- diag_mean** Model diagnostics for the final pre-weighted mean model.
- diag_var** Model diagnostics for the variance model.

See Also

Other Precipitation trend analysis: [classify_mean_variance_trends\(\)](#), [classify_trend\(\)](#), [estimate_by_estimate\(\)](#), [estimate_one_state_fgl\(\)](#), [extract_model_diagnostics\(\)](#), [fisher_joint_pvalue\(\)](#)

estimate_one_state_fgls

Estimate mean and variance trends for one state using an FGLS-style workflow

Description

Estimates a panel-data mean model, uses the residuals from that model to estimate a variance model, constructs inverse-variance weights, and then re-estimates the mean model after applying the weights directly to the data.

Usage

```
estimate_one_state_fgls(dt, formula, model = "within", eps = 0.00000001)
```

Arguments

dt	A data.table or data frame containing the state-level panel data. The data must include panelid, commodity_year, precipitation, trend, and county_fips.
formula	A model formula for the mean equation. In the current workflow, this is expected to be precipitation ~ trend:factor(county_fips).
model	Character string specifying the plm estimator. Common values include "within", "pooling", "random", "between", and "fd".
eps	Small positive value added to squared residuals and predicted variances to avoid taking $\log(0)$ or dividing by zero. Default is $1e-8$.

Details

The workflow proceeds in six steps:

1. Estimate an initial panel mean model.
2. Extract residuals from the initial mean model.
3. Construct a log-squared residual outcome: $\log_resid_sq = \log(residual^2 + eps)$.
4. Estimate a variance equation using the same RHS variables as the mean equation.
5. Predict the conditional variance and construct inverse-variance weights.
6. Multiply precipitation and trend by $\sqrt{weight}$ and re-estimate the mean model without using the weights argument in plm.

This pre-weighting strategy avoids the plm limitation that robust covariance functions such as `vcovDC()` and `vcovHC()` are not implemented for weighted panel regressions.

Value

A list containing:

mean_model_initial Initial fitted mean model.

variance_model Fitted residual-based log-variance model.

mean_model_weighted FGLS-style pre-weighted fitted mean model.

data Estimation sample used for the variance model.

data_weighted Pre-weighted estimation sample used for the final mean model.

Returns NULL if the state does not have enough panel units or time periods, or if any model estimation step fails.

Important

The pre-weighting block is currently hard-coded for the model:

```
precipitation ~ trend:factor(county_fips)
```

If the outcome variable or trend variable changes, the pre-weighting block must be updated accordingly.

See Also

Other Precipitation trend analysis: [classify_mean_variance_trends\(\)](#), [classify_trend\(\)](#), [estimate_by_estimated_trend\(\)](#), [estimate_one_state\(\)](#), [extract_model_diagnostics\(\)](#), [fisher_joint_pvalue\(\)](#)

estimate_turning_point_benchmark

Estimate Turning Points Relative to a Benchmark Window

Description

This function estimates the historical-window turning point for one or more groups in a dataset. The turning point is defined as the earliest value of an x-variable, such as the number of historical years, where the selected outcome remains within a specified tolerance band around a benchmark value for a required number of consecutive historical windows.

Usage

```
estimate_turning_point_benchmark(
  data,
  outcome,
  group_var,
  x_var,
  benchmark_x = NULL,
  tolerance_share = 0.05,
  consecutive_years = 5,
  require_consecutive_x = TRUE
)
```

Arguments

data	A data.frame or data.table containing the outcome variable, grouping variable, and historical-window variable.
outcome	Character string. Name of the outcome variable to evaluate. Examples include "actuarial_index", "alternative_lr", or "baseline_lr".
group_var	Character string. Name of the grouping variable used to estimate separate turning points. For example, "adjustment".

<code>x_var</code>	Character string. Name of the historical-window variable. This variable should represent the number of years in the historical window, such as 5, 6, ..., 60. The function coerces this variable to numeric.
<code>benchmark_x</code>	Numeric value indicating the x-value used as the benchmark. If NULL, the function uses the maximum observed value of <code>x_var</code> . For example, if the longest historical window is 60 years, use <code>benchmark_x = 60</code> . If the full sample is coded as 200, use <code>benchmark_x = 200</code> .
<code>tolerance_share</code>	Numeric value. The acceptable proportional distance from the benchmark value. The default is 0.05, meaning the outcome must be within 5 percent of the benchmark value.
<code>consecutive_years</code>	Integer. Number of consecutive historical-window values that must remain within the tolerance band before the first value is classified as the turning point. The default is 5.
<code>require_consecutive_x</code>	Logical. If TRUE, the function requires the selected sequence to be consecutive in terms of the actual <code>x_var</code> values. For example, 5, 6, 7, 8, 9 is valid, but 5, 6, 8, 9, 10 is not. If FALSE, the function only requires consecutive rows after sorting by <code>x_var</code> .

Details

The benchmark value is usually the outcome associated with the longest historical window, such as the 60-year window or the full-sample window. For each group, the function compares the outcome at each historical window to the group-specific benchmark value and identifies the earliest window at which the outcome is sufficiently close to the benchmark and remains close for the required number of consecutive windows.

For each group, the function computes the absolute distance between the outcome at historical window k and the benchmark value:

$$distance_k = |y_k - y_{benchmark}|$$

This distance is compared with a tolerance band:

$$tolerance = tolerance_share \times |y_{benchmark}|$$

The turning point is the first historical window k such that $distance_k \leq tolerance$ for `consecutive_years` consecutive historical windows.

If no such sequence is found for a group, the function returns NA for that group's turning point.

If duplicate observations exist for the same group and historical window, the function averages the outcome within that group-window combination before estimating the turning point.

Value

A data.table with one row per group. The output includes the grouping variable, outcome name, benchmark x-value, tolerance share, required number of consecutive years, and estimated turning point.

`extract_model_diagnostics`*Extract diagnostics from a panel model*

Description

Extracts basic model diagnostics from a fitted `plm` model object. The function returns sample size, panel dimensions, R-squared values, residual sum of squares, and root mean squared error.

Usage

```
extract_model_diagnostics(obj)
```

Arguments

`obj` A fitted `plm` model object.

Details

This helper is designed for `plm` objects. R-squared values are extracted from `summary(obj)$r.squared` when available. Residual-based diagnostics are computed directly from the model residuals. Panel dimensions are recovered from `plm::index(obj)`.

Value

A `data.table` with one row containing:

n_obs Number of observations used in the model.

n_panel Number of unique panel units.

n_years Number of unique time periods.

r_squared Model R-squared, if available.

adj_r_squared Adjusted R-squared, if available.

rss Residual sum of squares.

rmse Root mean squared error.

See Also

Other Precipitation trend analysis: [classify_mean_variance_trends\(\)](#), [classify_trend\(\)](#), [estimate_by_estimated_trend\(\)](#), [estimate_one_state\(\)](#), [estimate_one_state_fgls\(\)](#), [fisher_joint_pvalue\(\)](#)

fisher_joint_pvalue	<i>Compute Fisher combined p-values for paired tests</i>
---------------------	--

Description

Combines two p-values using Fisher's method. In this workflow, the two p-values correspond to the mean trend test and the variance trend test for the same county.

Usage

```
fisher_joint_pvalue(p1, p2)
```

Arguments

p1	Numeric vector of p-values from the first test.
p2	Numeric vector of p-values from the second test.

Details

Fisher's method uses:

$$\chi^2 = -2 * (\log(p1) + \log(p2))$$

With two tests, the reference distribution is chi-square with $2 * 2 = 4$ degrees of freedom. Missing, zero, or negative p-values return NA.

Value

A data.table with:

joint_fisher_stat Fisher test statistic.

joint_fisher_p_value Combined p-value from a chi-square distribution with 4 degrees of freedom.

See Also

Other Precipitation trend analysis: [classify_mean_variance_trends\(\)](#), [classify_trend\(\)](#), [estimate_by_estimated_trend\(\)](#), [estimate_one_state\(\)](#), [estimate_one_state_fgls\(\)](#), [extract_model_diagnostics\(\)](#)

get_official_prf_polygon	<i>Retrieve the official RMA PRF grid shapefile</i>
--------------------------	---

Description

Downloads (if necessary) and loads the official USDA RMA Pasture, Rangeland, and Forage (PRF) Rainfall Index (RI) grid shapefile from the USFarmSafetyNetLab GitHub repository. The shapefile is cached locally in the user's R cache directory for subsequent use.

Usage

```
get_official_prf_polygon(official_prf_polygon_zip_file_path = NULL)
```

Arguments

official_prf_polygon_zip_file_path

official_prf_polygon_zip_file_path.

The cached shapefile will be reused on subsequent calls, reducing redundant downloads.

Value

A [SpatVector](#) object containing the official PRF RI grid polygons. The field grid_id uniquely identifies each grid cell.

get_price_indices	<i>Build a price-received deflator (PPIPR) series relative to current_year</i>
-------------------	--

Description

Constructs a table used to deflate nominal FCIP monetary amounts to a common base year. Returns two columns, commodity_year and PPIPR, where PPIPR equals the year's price-received index divided by the index in current_year (so PPIPR(current_year) == 1).

Usage

```
get_price_indices(current_year = NULL)
```

Arguments

current_year Integer scalar. The base year used for normalization. The returned PPIPR equals 1 for this year.

Details**Data sources (from rfcipDemand):**

- nassSurveyPriceRecivedIndex (annual; expects commodity_year, index_for_price_recived).
- nassAgPriceMonthlyIndex (monthly U.S. agricultural price index; expects year, comm, index).

Synthesizing the current year (if missing in the annual table):

- Compute the arithmetic mean of the monthly index where comm == "Agricultural" for both current_year and current_year - 1.
- Multiply last year's annual index_for_price_recived by the ratio mean_monthly(current_year) / mean_monthly(current_year - 1) to derive the current-year annual index.
- Append this row with data_source = "calculated".

Normalization:

- Let the denominator be the (mean) index_for_price_recived among rows with commodity_year == current_year (provides stability if duplicates exist).
- Define PPIPR = index_for_price_recived / denominator.

Output shape:

- Returns only commodity_year and PPIPR, sorted ascending by commodity_year.
- If the input annual table contains multiple rows per year, duplicates are preserved in the output (each with its own PPIPR). Aggregate if you require strictly one row per year (see Notes).

Value

A data.table with two columns:

- commodity_year - integer year.
- PPIPR - numeric deflator equal to the year's price-received index divided by the current_year index.

Assumptions & Notes

- Assumes both reference datasets from **rfcipDemand** are available with the specified columns (including the source's spelling index_for_price_recived).
- Monthly means are computed with na.rm = TRUE.
- If you need one row per year, post-aggregate: dt[, .(PPIPR = mean(PPIPR, na.rm = TRUE)), by = commodity_year].

get_study_releases	<i>Download all assets from a GitHub release with rate limiting</i>
--------------------	---

Description

Downloads all files attached to a specified GitHub release tag while **throttling requests** to avoid GitHub rate limits and abuse protection. This helper is designed for releases containing many or large assets (e.g., .rds outputs generated on HPC systems).

Usage

```
get_study_releases(
  owner,
  repository,
  release_tag,
  output_directory = NULL,
  github_token = NULL,
  sleep_seconds = 3,
  max_rounds = 3
)
```

Arguments

owner	Character string giving the GitHub repository owner (e.g., "ftsiboe").
repository	Character string giving the GitHub repository name (e.g., "indexDesignWindows").
release_tag	Character string specifying the GitHub release tag whose assets should be downloaded.
output_directory	Optional character string specifying the local directory where release assets should be saved. Defaults to data-raw/releases/{release_tag}.

github_token	Optional GitHub personal access token (PAT). Passed to piggyback via . token. Strongly recommended.
sleep_seconds	Numeric scalar giving the number of seconds to pause between individual file downloads. Increasing this value reduces the likelihood of triggering GitHub rate limits.
max_rounds	Integer giving the maximum number of retry rounds. Each round attempts to download any files still missing locally.

Details

The function downloads assets incrementally, pauses between requests, and retries failed downloads across multiple rounds. Already-downloaded files are skipped, allowing the function to safely resume after interruptions or rate-limit errors.

The function:

1. Constructs a default output directory (data-raw/releases/{release_tag}) if none is supplied.
2. Queries GitHub once to obtain the list of release assets.
3. Downloads assets **one at a time** using **piggyback**.
4. Pauses for sleep_seconds between downloads to reduce request bursts.
5. Retries failed or missing downloads for up to max_rounds.
6. Skips files that already exist locally.

This approach is especially useful when GitHub returns repeated HTTP 403 (Forbidden) errors during bulk downloads.

Authentication via a GitHub personal access token (PAT) is strongly recommended, even for public repositories.

Value

Invisibly returns NULL. Files are downloaded for their side effects.

plot_fcip_main_outcomes

Plot FCIP Main Outcomes (Faceted Bar Charts)

Description

The function expects data to contain:

- a numeric/factor year column (specified via colume_year);
- a numeric value column (bar heights);
- a categorical "outcome" column (specified via colume_outcome) used for faceting; and
- an optional grouping column (specified via colume_grouping) used to color/ fill bars and determine legend entries.

For legend ordering, the function ranks groups using the subset of rows from the most recent year *within the facet whose label equals "(B) Liability amount in U.S. dollars"*. If your outcome labels differ, you may want to harmonize them or adjust the code that builds labs.

Usage

```
plot_fcip_main_outcomes(
  data,
  colume_outcome,
  colume_year,
  colume_grouping = NULL,
  time_scale_theme = NULL,
  general_theme = NULL,
  palette = c("#003524", "#00583D", "#A0BD78", "#BED73B", "#D7E5C8", "#FEF389",
    "#FFC425", "#BE5E27", "#9DD9F7", "#51ABA0", "#0F374B")
)
```

Arguments

<code>data</code>	A <code>data.frame</code> (or <code>data.table</code>) with at least the columns: <code>value</code> , the column named by <code>colume_year</code> , the column named by <code>colume_outcome</code> , and (optionally) the column named by <code>colume_grouping</code> .
<code>colume_outcome</code>	Character scalar. Name of the column in <code>data</code> that defines facet panels (e.g., outcome labels such as "(A) Net reported acres" and "(B) Liability amount in U.S. dollars").
<code>colume_year</code>	Character scalar. Name of the year column in <code>data</code> (x-axis, typically the FCIP commodity year).
<code>colume_grouping</code>	Character scalar or <code>NULL</code> . Name of the grouping column in <code>data</code> used for fill/color and legend entries. If <code>NULL</code> , all bars are treated as a single group.
<code>time_scale_theme</code>	Optional <code>ggplot2</code> scale/theme element controlling the x-axis (e.g., <code>scale_x_continuous(...)</code>). If <code>NULL</code> , a default scale is applied using the unique values of the supplied year column.
<code>general_theme</code>	Optional <code>ggplot2</code> theme applied to the figure. If <code>NULL</code> , <code>ers_theme()</code> is used and further tweaked (text sizes, legend layout, facet strips). Requires that <code>ers_theme()</code> is available in scope.
<code>palette</code>	Optional character vector of HEX colors used by <code>scale_fill_manual()</code> . If provided, it is applied to the grouping legend.

Details

Creates a faceted bar chart of FCIP outcomes over time by a chosen grouping, with a shared x-axis (commodity year) and a legend. Facets are determined by an "outcome" column in `data` (e.g., liability vs. acres), allowing one or more panels depending on the unique values present.

Internally, the function:

1. Copies the columns named by `colume_outcome`, `colume_grouping`, and `colume_year` into standard placeholders used in aesthetics.
2. Builds a ranking of groups from the latest year within the facet whose label equals "(B) Liability amount in U.S. dollars", then maps that ranking to legend order.
3. Draws a bar chart of `value ~ year`, colored/filled by group, and `facet_wrap()`s by outcome (free y-scales).
4. Applies the supplied or default x-axis scale and theme, plus optional manual fill palette.

Value

A ggplot object.

Note

The parameter names use `colume_*` (typo preserved for backward compatibility). Consider aliasing/renaming in a future version.

plot_liability_and_acres

Plot Liability and Net Reported Acres Faceted by a Grouping Variable

Description

Produces a two-panel bar chart:

1. **Panel A** - Liability (in U.S.\ dollars) by year and group.
2. **Panel B** - Net reported acres (in acres) by year and group.

The panels share the same x-axis (commodity years) and a single legend that is displayed beneath the figure.

Usage

```
plot_liability_and_acres(
  data,
  grouping_variable,
  grouping_name = NULL,
  time_scale_theme = NULL,
  general_theme = NULL,
  label_liability = "Billion U.S. dollars\n",
  label_net_reported_acres = "Million acres\n",
  palette = c("#003524", "#00583D", "#A0BD78", "#BED73B", "#D7E5C8", "#FEF389",
    "#FFC425", "#BE5E27", "#9DD9F7", "#51ABA0", "#0F374B")
)
```

Arguments

<code>data</code>	A data.frame containing at least these columns: • <code>commodity_year</code> - numeric or factor; the x-axis. • <code>value</code> - numeric; the bar heights. • <code>variable</code> - factor with the levels "(A) Liability in U.S. dollars" and "(B) Net reported acres". • The column referenced by <code>grouping_variable</code>.
<code>grouping_variable</code>	A character scalar giving the name of the column in data that defines the groups (e.g. "commodity", "state").
<code>grouping_name</code>	Optional character scalar used as the legend title. Defaults to the value of <code>grouping_variable</code> if NULL.

time_scale_theme	Optional ggplot2 scale or theme element that controls the x-axis breaks/labels. If NULL, the function applies <code>scale_x_continuous(breaks = unique(data\$commodity_year), labels = unique(data\$commodity_year))</code> .
general_theme	Optional ggplot2 theme applied to both panels. If NULL, <code>ers_theme()</code> is used with additional tweaks for font sizes, legend layout, and facet strips.
label_liability	Y-axis label for Panel A. Default "Billion U.S. dollars\\n".
label_net_reported_acres	Y-axis label for Panel B. Default "Million acres\\n".
palette	A character vector of hex colors used to fill the bars. The default is an 11-color palette aligned with ERS/NDSU branding.

Details

Internally the function:

1. Duplicates the column named by `grouping_variable` into `data$grouping_variable` for convenient aesthetic mapping.
2. Builds two separate ggplot bar charts (one per variable level), applies user themes and palettes, and hides their legends.
3. Extracts a shared legend and x-axis grob with `gtable_filter()`.
4. Assembles everything with `gridExtra::grid.arrange()`.

Value

A named list with five objects:

`fig` A gtable containing the assembled two-panel figure, shared x-axis label, and bottom legend.

`figA` The ggplot object for Panel A.

`figB` The ggplot object for Panel B.

`ldgnd` The extracted legend as a gtable.

`xlabT` The shared x-axis grob as a gtable.

Examples

```
## Not run:
# Group by commodity with a custom x-axis theme
plot_liability_and_acres(
  data = data_comm,
  grouping_variable = "commodity",
  grouping_name = "Commodity",
  time_scale_theme = ggplot2::scale_x_continuous(
    breaks = seq(2008, 2024, 2),
    labels = seq(2008, 2024, 2)
  ),
  general_theme = ggplot2::theme_minimal()
)

## End(Not run)
```

plot_prf_statistics *Plot PRF Summary Statistics on the Official Grid*

Description

Creates a choropleth map of Pasture, Rangeland, Forage (PRF) statistics on the official PRF grid. Optionally facets the map by a disaggregate variable (e.g., data source: Official vs. Study Approximation).

Usage

```
plot_prf_statistics(
  data,
  outcome_variable,
  plot_title,
  prf_polygon = NULL,
  disaggregate_variable = NULL,
  spatial_unit = "prf_grid",
  palette = c("#FEF9C5", "#FFC425", "#FEF389", "#E7F2B4", "#D7E5C8", "#C5DE91",
    "#BED73B", "#A0BD78", "#00583D", "#0F374B"),
  na.value = "white"
)
```

Arguments

data	A data frame containing at least <code>grid_id</code> and the variable specified in <code>outcome_variable</code> . If <code>disaggregate_variable</code> is used, it must also be present in data.
outcome_variable	Character scalar giving the name of the variable in data to map (e.g., a binned category such as "mean_cat").
plot_title	Character scalar used as the main plot title.
prf_polygon	Optional <code>sf</code> object containing PRF grid polygons with a <code>grid_id</code> column. If <code>NULL</code> , the function calls <code>get_official_prf_polygon()</code> .
disaggregate_variable	Optional character scalar giving the name of a column in data used for faceting (e.g., "disaggregate" or "source"). If <code>NULL</code> , no facets are drawn.
spatial_unit	Character scalar. One of "prf_grid" or "county". Determines which polygons are used and which join key is required.
palette	Character vector of hex color codes used for the fill scale. The default is a 10-color sequential/diverging palette tailored to PRF maps.
na.value	Fill color for the background (non-data areas) and missing values.

Details

The function:

1. Loads the official PRF grid polygons via `get_official_prf_polygon()` if `prf_polygon` is not supplied.
2. Filters PRF polygons to those with `grid_id` present in data.

3. Transforms the PRF polygons to match the CRS of the state boundaries from `urbnmapr::get_urbn_map(map = "states")`.
4. Joins data to the PRF polygons by `grid_id`.
5. Filters out rows with missing values in `outcome_variable`.
6. Plots a choropleth map using the supplied palette, overlaying U.S. state boundaries (excluding Alaska and Hawaii; FIPS 2 and 15).
7. Optionally facets the plot by `disaggregate_variable`, with two columns of panels.

The function assumes that data contains a column named `grid_id` matching the PRF grid IDs, and a column whose name is given by `outcome_variable`. When `disaggregate_variable` is provided, it should also be a column in data.

Value

A ggplot object representing the PRF statistics map, optionally with facets by `disaggregate_variable`.

Examples

```
## Not run:
p <- plot_prf_statistics(
  data          = df,
  outcome_variable = "mean_cat",
  plot_title     = "PRF Index Mean (2016-2024)",
  disaggregate_variable = "disaggregate"
)
print(p)

## End(Not run)
```

plot_us_states_choropleth

Plot a US States Choropleth With State-Level Labels

Description

Builds a U.S. states choropleth by joining user-supplied state data (by FIPS) to the `urbnmapr` states basemap. States are filled using a categorical column, and each state is labeled with a two-line label (state abbreviation + a value column). Labels for small states are placed with repelled text to reduce overlap, with special nudges for Vermont and New Hampshire. Optionally, a table grob can be embedded in the lower-left of the map.

Usage

```
plot_us_states_choropleth(
  data,
  value_lable,
  category_lable,
  legend_title = NULL,
  state_text_colors = c("black", "black"),
  palette = c("#BE5E27", "#FFC425", "#FEF389", "#BED73B", "#A0BD78", "#00583D",
```

```

    "#003524", "#D7E5C8", "#9DD9F7", "#51ABA0", "#0F374B"),
  table_grob = NULL,
  label_size = 2.5,
  na.value = "white",
  keep_all_states = FALSE
)

```

Arguments

<code>data</code>	A <code>data.frame</code> containing (at minimum) a <code>state_code</code> column of numeric state FIPS codes used to join to the <code>urbnmapr</code> states map. It must also contain the columns referenced by <code>value_label</code> and <code>category_label</code> .
<code>value_label</code>	Character scalar. Name of the column in <code>data</code> used to populate the numeric/value portion of the state label (second line). Values are inserted as-is (not rounded/formatted) and are coerced through <code>as.data.frame(sf_object)[, value_label]</code> .
<code>category_label</code>	Character scalar. Name of the column in <code>data</code> used for choropleth fills. Rows with missing values in this column are dropped unless you set <code>keep_all_states = TRUE</code> (which still draws all states, but only colors those present after filtering).
<code>legend_title</code>	Character. Legend title for the fill scale. If <code>NULL</code> , the title is set to an empty string (i.e., no visible title).
<code>state_text_colors</code>	Character vector of length 2. The first color is used for labels on non-small states; the second color is used for repelled labels on small states.
<code>palette</code>	Character vector of hex colours used by <code>scale_fill_manual()</code> for the categories in <code>category_label</code> .
<code>table_grob</code>	Optional grob to embed on the map using <code>annotation_custom()</code> (e.g., <code>gridExtra::tableGrob(...)</code>). If <code>NULL</code> , no table is added.
<code>label_size</code>	Numeric. Text size passed to <code>geom_sf_text()</code> and <code>geom_text_repel()</code> .
<code>na.value</code>	Character. Fill color used for missing values when drawing the basemap (and as <code>na.value</code> in <code>scale_fill_manual()</code>).
<code>keep_all_states</code>	Logical. If <code>TRUE</code> , first draws the full U.S. states basemap in <code>na.value</code> (so states not present in <code>data</code> remain visible). If <code>FALSE</code> , only the joined/filtered states are drawn.

Details

The function:

- Loads the states basemap via `urbnmapr::get_urbn_map(map = "states", sf = TRUE)` and creates a numeric `state_code` from `state_fips`.
- Left-joins `data` on `state_code`, then filters out rows where `category_label` is `NA`.
- Builds a two-line label of `STATE_ABB` plus the `value_label` column.
- Flags "small" states as those with area < 50,000 km² (computed after transforming to EPSG:5070 for equal-area calculations).
- Uses `geom_sf_text()` for non-small states and `ggrepel::geom_text_repel()` for small states, nudging labels left/right based on whether the centroid lies east or west of the map midpoint; Vermont and New Hampshire get a custom nudge.
- Optionally adds `table_grob` at fixed map coordinates using `annotation_custom()`.

Value

A ggplot object (choropleth of U.S. states) with state labels and an optional embedded table grob.

policytime	<i>Add U.S. Farm Policy Vertical Lines and Labels to a ggplot</i>
------------	---

Description

This function overlays vertical lines and text labels on a ggplot object to mark major U.S. agricultural policy events (e.g., Farm Bills or Acts). The vertical lines are drawn at specific years, and labels are positioned based on values provided in the `pty` vector.

Usage

```
policytime(pty, plot, size)
```

Arguments

<code>pty</code>	A numeric vector of y-axis positions for label placement. The length of the vector determines which policy annotations are added: • 1st: 1980 Act • 2nd: 1994 Act • 3rd: 1996 Farm Bill • 4th: 2000 Agricultural Risk Protection Act • 5th: 2008 Farm Bill • 6th: 2014 Farm Bill • 7th: 2018 Farm Bill
<code>plot</code>	A ggplot object to which policy lines and labels will be added.
<code>size</code>	A numeric value indicating the text size of the policy labels.

Details

The lines and labels are added in brown color with dashed lines (`lty=5`), and labels are rotated vertically. Labels are only drawn if the corresponding index exists in `pty`.

Value

A ggplot object with added vertical dashed lines and corresponding text labels for each policy year provided.

Examples

```
## Not run:
base_plot <- ggplot(data, aes(x = commodity_year, y = value)) +
  geom_line()
policy_positions <- c(10, 15, 20, 25, 30, 35, 40)
annotated_plot <- policytime(policy_positions, base_plot, size = 3)
print(annotated_plot)

## End(Not run)
```

reshape_statistics	<i>Reshape and Label Balance Statistics</i>
--------------------	---

Description

Reshapes selected balance statistics from a data frame into long format and assigns descriptive labels to each statistic disaggregate. This function is commonly used to prepare datasets for balance comparison plots or summary tables.

Usage

```
reshape_statistics(
  data,
  statistic_variables,
  disaggregate_labels,
  rename = NULL,
  panel_variables
)
```

Arguments

<code>data</code>	A data frame containing <code>grid_id</code> and the statistic variables to be reshaped.
<code>statistic_variables</code>	Character vector of column names in <code>data</code> that contain the statistics to reshape.
<code>disaggregate_labels</code>	Character vector of descriptive labels corresponding to <code>statistic_variables</code> . Must be the same length and in the same order.
<code>rename</code>	Optional character scalar. If supplied, the output column value will be renamed to this string (e.g., "std_diff" or "index").
<code>panel_variables</code>	panel variables

Details

The function:

1. Validates that `statistic_variables` and `disaggregate_labels` have equal length.
2. Checks that all specified statistic variables exist in `data`.
3. Selects `grid_id` and the variables listed in `statistic_variables`.
4. Converts the data from wide to long format using `tidyr::gather()`, creating columns: `grid_id`, `disaggregate`, and `value`.
5. Converts `disaggregate` to a factor and replaces raw column names with human-readable labels using `disaggregate_labels`.
6. Optionally renames the `value` column using the `rename` argument.

Value

A long-format data frame with the columns:

grid_id Identifier for the spatial/grid unit.

disaggregate Factor indicating the origin of each statistic, labeled using disaggregate_labels.

value Numeric value of the statistic, or renamed to rename if provided.

```
select_tolerance_by_post_stability
```

Select a Data-Driven Tolerance Using Post-Turning-Point Stability

Description

This function selects a tolerance value for the benchmark-distance turning-point method by evaluating a grid of candidate tolerance values. For each tolerance, it first estimates the turning point using `estimate_turning_point_benchmark()`. It then measures how stable the outcome is after the estimated turning point.

Usage

```
select_tolerance_by_post_stability(
  data,
  outcome,
  group_var,
  x_var,
  benchmark_x = NULL,
  tolerance_grid = seq(0.01, 0.15, by = 0.005),
  consecutive_years = 5,
  penalty_weight = 0.01,
  require_consecutive_x = TRUE
)
```

Arguments

data A data.frame or data.table containing the outcome variable, grouping variable, and historical-window variable.

outcome Character string. Name of the outcome variable used to evaluate stability. Examples include "actuarial_index", "alternative_lr", or "baseline_lr".

group_var Character string. Name of the grouping variable. Separate tolerance values and turning points are selected for each group.

x_var Character string. Name of the numeric historical-window variable. For example, "history_years".

benchmark_x Numeric. The x-value used as the long-history benchmark in `estimate_turning_point_benchmark()`. If NULL, the maximum observed value of `x_var` is used.

tolerance_grid Numeric vector. Candidate tolerance shares to evaluate. The default evaluates values from 1 percent to 15 percent in increments of 0.5 percentage points.

consecutive_years Integer. Number of consecutive historical-window values that must remain within the benchmark-distance tolerance before the first x-value is classified as the turning point. The default is 5.

penalty_weight Numeric. Penalty applied to later turning points in the score function. Larger values favor earlier turning points, while smaller values allow later turning points if they produce lower post-turning-point variability. The default is 0.01.

require_consecutive_x

Logical. If TRUE, the function requires the selected sequence to be consecutive in terms of the actual x_var values. This argument is passed to estimate_turning_point_benchmark()

Details

The selected tolerance is the one that minimizes a normalized stability score:

$$score = post_cv + penalty_weight \times scaled_turning_point$$

where post_cv is the post-turning-point coefficient of variation and scaled_turning_point is the estimated turning point divided by the benchmark x-value. This normalization makes the score more comparable across outcomes with different units or scales.

This function is designed to make the tolerance parameter in the benchmark turning-point method more data-driven. Instead of imposing a fixed tolerance, such as 5 percent, the function searches over a grid of possible tolerances.

For each candidate tolerance:

1. A turning point is estimated using estimate_turning_point_benchmark().
2. The data are restricted to observations at or after the estimated turning point.
3. The post-turning-point mean, standard deviation, coefficient of variation, and range of the outcome are calculated.
4. A normalized score is computed using post-turning-point variability and a penalty for later turning points.

The tolerance with the lowest score is selected for each group.

If no valid turning point is found for a candidate tolerance, that candidate does not contribute to the post-turning-point stability calculation for that group.

Value

A data.table with one row per group. The output includes the selected tolerance share, estimated turning point, post-turning-point mean, standard deviation, coefficient of variation, range, scaled turning point, and the minimized score.

setup_environment	Setup Project Environment
-------------------	---------------------------

Description

Initializes the working environment for a project by creating required directories, setting useful global options, and fixing the random seed.

Usage

```
setup_environment(
  year_beg = 2001,
  year_end = as.numeric(format(Sys.Date(), "%Y")),
  seed = 1980632,
  project_name,
  local_directories = list(file.path("data-raw", "output"), file.path("data-raw",
    "scripts"), file.path("data")),
  fastscratch_root = NULL,
  fastscratch_directories = NULL
)
```

Arguments

year_beg Integer. Beginning year of the analysis (default: 2001).

year_end Integer. Ending year of the analysis (default: current system year).

seed Integer. Random seed for reproducibility (default: 1980632).

project_name Character. Project name (required). Used to build fast-scratch directory paths.

local_directories List of project-local directories to create (default: list("data-raw/output", "data-raw/scripts", "data")).

fastscratch_root Optional character. Root directory for fast-scratch files. If NULL, it is set automatically:

- Windows: "C:/fastscratch"
- Linux/macOS: "/fastscratch/<username>"

fastscratch_directories List of fast-scratch subdirectories (relative to <fastscratch_root>/<project_name>) to create. If NULL, no fast-scratch subdirectories are created and wd is returned as an empty list.

Details

The function ensures the requested directories exist, creating them if necessary. Directory keys in the returned wd list are the basenames of the provided fastscratch_directories.

It also sets the following options:

- options(scipen = 999) (turns off scientific notation)
- options(future.globals.maxSize = 8 * 1024^3) (~8 GiB)
- options(dplyr.summarise.inform = FALSE) (quiet **dplyr**)

Finally, the random number generator is seeded with the provided seed.

Value

A list with:

wd Named list of created fast-scratch directories. Empty if fastscratch_directories = NULL.

year_beg Starting year (integer).

year_end Ending year (integer).

seed Seed value used for RNG.

Index

* Precipitation trend analysis

- classify_mean_variance_trends, [3](#)
- classify_trend, [4](#)
- estimate_by_estimator, [7](#)
- estimate_one_state, [10](#)
- estimate_one_state_fgls, [11](#)
- extract_model_diagnostics, [14](#)
- fisher_joint_pvalue, [15](#)

add_break_categories, [2](#)

classify_mean_variance_trends, [3](#), [5](#), [8](#),
[10](#), [12](#), [14](#), [15](#)

classify_trend, [4](#), [4](#), [8](#), [10](#), [12](#), [14](#), [15](#)

compute_balance, [5](#)

ers_theme, [7](#)

estimate_by_estimator, [4](#), [5](#), [7](#), [10](#), [12](#), [14](#),
[15](#)

estimate_final_turning_point, [8](#)

estimate_one_state, [4](#), [5](#), [8](#), [10](#), [12](#), [14](#), [15](#)

estimate_one_state_fgls, [4](#), [5](#), [8](#), [10](#), [11](#),
[14](#), [15](#)

estimate_turning_point_benchmark, [12](#)

extract_model_diagnostics, [4](#), [5](#), [8](#), [10](#), [12](#),
[14](#), [15](#)

fisher_joint_pvalue, [4](#), [5](#), [8](#), [10](#), [12](#), [14](#), [15](#)

get_official_prf_polygon, [15](#)

get_price_indices, [16](#)

get_study_releases, [17](#)

plot_fcip_main_outcomes, [18](#)

plot_liability_and_acres, [20](#)

plot_prf_statistics, [22](#)

plot_us_states_choropleth, [23](#)

policytime, [25](#)

reshape_statistics, [26](#)

select_tolerance_by_post_stability, [27](#)

setup_environment, [28](#)

SpatVector, [16](#)